

Service Lifecycle Operations

CYBOL Operation \ Channel	file directory	serial port	terminal	named pipeline	client socket	server socket	display (xcb)	display (win32)	mobile app
maintain/startup (server)									
configure/initialise (server config)	-	-	-	-	-	+	+	+	
list/attach (server entry, client list)	-	-	-	-	-	+	+	+	
attend/enable (accept thread)	-	-	-	-	-	+(handler)	+(global event loop)	-	
maintain/startup (client)									
dispatch/open (client device config)	+	+	+	+	+	+(in enable handler)	+(window id)	+(window id)	
buffer/activate (client entry, async)	-	o (lazy init list/attach)	o (lazy init list/attach)	o (lazy init list/attach)	o (lazy init list/attach)	+(in enable handler)	+	+	
feel/sense (message thread)	-	o	o	o	o	+(handler)	+(handler, NO thread)	+(handler, USE thread)	
communicate/receive (data)									
stream/read	+	+	+	+	+	+	+	+	
pack/extract	o	o	o	o	o	o	-	-	
convert/decode	o	o	o	o	o	o	-	-	
represent/deserialise	o	o	o	o	o	o	+	+	
(process data in various ways)									
represent/serialise	o	o	o	o	o	o	+	+	
convert/encode	o	o	o	o	o	o	-	-	
pack/compress	o	o	o	o	o	o	-	-	
stream/write	+	+	+	+	+	+	+	+	
communicate/send (data)									
feel/suspend	-	o	o	o	o	+	+	+	
buffer/deactivate	-	o	o	o	o	+	+	+	
dispatch/close	+	+	+	+	+	+	+	+	
communicate/send (client)									
attend/disable	-	-	-	-	-	+	+	-	
list/cutoff	-	o	o	o	o	+	+	+	
configure/finalise	-	-	-	-	-	+	+	+	
maintain/shutdown (server)									
The yellow are convenience methods	summing up several others, for easy usage.								
Properties of maintain/startup:									
flag „server“ on „true“: calls configure + list + attend									
flag „server“ on „false“ (client): calls dispatch									
flag „asynchronous“ on „true“: calls buffer + feel									

Service Lifecycle Operations

Display:
open each window and buffer manually
catch events of ALL windows using global function of window system
get id of window that the event stems from
assign event to correct buffer of the corresponding client window id in cyboi
use irq pipe to inform main thread
read sender (client window id) from irq pipe in order to assign correct handler
use client window id in handler in order to use the correct buffer of the client entry
The feel/sense operation may be called for each window separately, in order to assign a specific handler (corresponds to the controller in MVC pattern).
However, a new thread MUST NOT be started for the feel/sense function, since the global activate/enable catches all events.
This can be achieved by distinguishing channels in the feel/sense cyboi function, to NOT create a thread for display.
Serial Port and Terminal Client:
They may be used as standalone client, so that a call of dispatch/open and stream/write is enough for synchronous (blocking) communication.
However, for asynchronous communication the buffer/activate and feel/sense need to be called as well.
The client entry allocated by buffer/activate has to be stored in a server entry, so that it can be retrieved again later.
Therefore, list/attach is called automatically inside cyboi, so that a server entry gets allocated.
The feel/sense takes a handler as cybol property.
Actually, dispatch/open and buffer/activate could call feel/sense automatically in cyboi.
But there are terminals that use direct blocking input and do not need a sensing thread at all.
Other terminals are just for output and do not need any input at all.
Therefore, the buffer/activate and feel/sense operations may be called optionally, but separately, after dispatch/open.
They are separate operations and not combined so that the thread may be suspended temporarily without having to deallocate buffer resources.
Server Socket:
The client socket id that was accepted in attend/enable gets handed over via pipe to the cyboi main thread and there it gets assigned to the cybol property „sender“.
Within the attend/enable handler, the dispatch/open and feel/sense operations may relate to this cybol property „sender“, which is the client socket id.
Asynchronous Communication (bufferer and feeler):
The operations buffer/activate and feel/sense are only necessary, if asynchronous communication is used.
In this case, buffer/open will provide two internal buffers, choosable by two flags: one for input and another one for output.
With synchronous (direct) communication, they are not needed and communicate/receive or communicate/send receive data directly from cybol.
Example: Communication via terminal with standard blocking input/output.
With asynchronous (indirect) communication, data are written to and/or read from buffer.
It is usable without server and client list, if cybol manages and hands over the client id to each cybol operation, so that the correct buffer may be used.
The „asynchronous“ flag needs to be handed over to read and write, so that they can decide whether to read directly (sync) or from buffer (async).
Multiclient Communication (lister and attender):
The operations are only necessary, if many clients are to be managed by cyboi.
Example: Communication via server socket accepting many requests.
For this case, cyboi provides internal client lists keeping resources and timeout numbers.
Without multiclient communication, the cybol application is responsible for managing client identification numbers, to pass them to each send or receive.